

Mundo do Aspirador de Pó em Python

Guia completo para iniciantes: agentes reativos e agentes baseados em modelo

Este material acompanha o código do PDF “**Simulação Comparativa de Agentes Inteligentes: Reativo Simples vs. Baseado em Modelo**”. O objetivo é explicar, do zero, como construir, executar, entender e modificar a simulação.

Sumário

1. [Introdução](#)
 2. [Preparação do ambiente](#)
 3. [Código completo](#)
 4. [Construção do código, bloco por bloco](#)
 5. [Conceitos-chave](#)
 6. [Como testar e modificar](#)
 7. [Desafios propostos](#)
 8. [Glossário](#)
 9. [Resumo final](#)
 10. [Referências](#)
-

1. Introdução

1.1 O que é o problema do Mundo do Aspirador de Pó?

O **Mundo do Aspirador de Pó** é um problema clássico de Inteligência Artificial. Imagine um robô aspirador dentro de uma casa dividida em quadrados. Alguns quadrados estão limpos, outros têm sujeira e alguns estão bloqueados por paredes ou móveis.

O robô precisa observar o local onde está, escolher uma ação e receber uma consequência. Ele pode, por exemplo:

- aspirar a sujeira da célula atual;

- mover-se para o norte, sul, leste ou oeste;
- permanecer parado quando não houver uma ação possível.

A casa é representada por uma **grade**, também chamada de matriz bidimensional. Uma grade `5 x 5`, por exemplo, tem cinco linhas e cinco colunas, totalizando 25 células.

A pergunta central da simulação é:

Qual agente consegue limpar mais sujeira e gastar menos pontos ao longo de uma quantidade limitada de passos?

O programa compara dois comportamentos:

Agente	Forma de decidir	Possui memória?
Reativo simples	Segue regras baseadas apenas no que percebe agora	Não
Baseado em modelo	Usa a percepção atual e uma matriz de visitas	Sim

1.2 O que são agentes inteligentes?

Um **agente** é algo que percebe um ambiente por meio de sensores e atua nesse ambiente por meio de atuadores.

Um robô aspirador real é um exemplo intuitivo:

- seus sensores detectam sujeira, paredes e obstáculos;
- seu sistema de decisão escolhe o próximo movimento;
- seus motores movimentam as rodas;
- seu aspirador remove a sujeira.

Neste programa, o agente não é um robô físico. Ele é um objeto Python que recebe um dicionário de percepções e devolve uma ação.

O ciclo fundamental é:

Plain Text

perceber → decidir → agir → atualizar o ambiente

Esse ciclo se repete até atingir o número máximo de passos.

1.3 O formalismo PEAS

O formalismo **PEAS** descreve um agente por quatro elementos:

- **P — Performance**, ou desempenho;
- **E — Environment**, ou ambiente;
- **A — Actuators**, ou atuadores;
- **S — Sensors**, ou sensores.

Performance: desempenho

A pontuação indica se as decisões foram vantajosas. Nesta simulação:

Evento	Pontuação
Aspirar uma célula suja	+10
Fazer um movimento válido	-1
Colidir com obstáculo ou borda	-5
Aspirar uma célula vazia	0
Permanecer parado	0

A limpeza recebe uma recompensa grande. O deslocamento tem um pequeno custo, representando energia ou combustível. A colisão recebe uma penalidade maior para desencorajar movimentos inválidos.

Environment: ambiente

O ambiente é uma grade $N \times N$ com:

- células vazias;
- células com sujeira;
- obstáculos;
- posição atual do agente;
- pontuação acumulada.

A distribuição inicial de sujeira e obstáculos é aleatória, mas pode ser reproduzida usando uma **seed**.

Actuators: atuadores

Os atuadores são as ações que o agente pode executar:

- aspirar;
- mover para norte;
- mover para sul;
- mover para leste;
- mover para oeste;
- permanecer parado.

Sensors: sensores

Os sensores informam:

- se existe sujeira na célula atual;
- se cada uma das quatro células vizinhas é navegável;
- a posição atual do agente.

1.4 Objetivo da simulação

O objetivo é comparar duas arquiteturas de agentes sob condições equivalentes. Os ambientes são criados com a mesma seed, portanto começam com a mesma distribuição de obstáculos e sujeira. Os agentes também começam na mesma posição.

O **agente reativo simples** não sabe por onde já passou. Se não encontrar sujeira, escolhe aleatoriamente uma saída livre. Por isso, pode voltar muitas vezes aos mesmos lugares.

O **agente baseado em modelo** mantém uma matriz de visitas. Ele prefere, localmente, uma célula vizinha que tenha sido visitada menos vezes. Essa estratégia tende a melhorar a cobertura espacial, embora não garanta o melhor caminho global.

2. Preparação do ambiente

2.1 Instalar Python

Recomenda-se usar **Python 3.10 ou mais recente**. O Python pode ser baixado no site oficial: python.org.

Durante a instalação no Windows, marque a opção **Add Python to PATH**. Depois, abra um terminal e confirme:

Bash

```
python --version
```

Em alguns sistemas Linux e macOS, o comando é:

Bash

```
python3 --version
```

2.2 Bibliotecas necessárias

O programa usa:

Biblioteca	Uso	Instalação adicional
tkinter	Interface gráfica	Geralmente vem com Python; em algumas distribuições Linux precisa ser instalada pelo sistema
numpy	Matrizes e números aleatórios	Sim
random	Escolhas aleatórias simples	Não; faz parte da biblioteca padrão
typing	Anotações de tipos	Não; faz parte da biblioteca padrão

Instale NumPy com:

Bash

```
python -m pip install numpy
```

No Linux baseado em Debian ou Ubuntu, o Tkinter pode ser instalado com:

Bash

```
sudo apt update  
sudo apt install python3-tk
```

No macOS, a instalação oficial do Python normalmente inclui Tkinter. No Windows, a instalação padrão também costuma incluí-lo.

Para verificar Tkinter:

Bash

```
python -c "import tkinter; print('Tkinter funcionando')"
```

Para verificar NumPy:

Bash

```
python -c "import numpy; print(numpy.__version__)"
```

O módulo `random` não deve ser instalado com `pip`, pois já pertence ao Python.

2.3 Criar e executar o arquivo

Crie uma pasta para o projeto:

Bash

```
mkdir mundo_aspirador  
cd mundo_aspirador
```

Crie um arquivo chamado:

Plain Text

`mundo_aspirador.py`

Cole nele o código da próxima seção. Execute com:

Bash

```
python mundo_aspirador.py
```

Ou, em sistemas que usam `python3`:

Bash

```
python3 mundo_aspirador.py
```

Uma janela será aberta com duas grades, botões de controle e dois painéis de log.

3. Código completo

O código abaixo é a versão organizada e executável do código do PDF. Durante a extração do PDF, algumas quebras de página apareceram no meio de linhas e alguns níveis de indentação ficaram visualmente deslocados. Essas quebras foram corrigidas aqui.

Também foi removido o import `tkk`, porque ele não é usado no programa. A lógica original foi preservada.

Python

```
"""Simulação comparativa de agentes inteligentes.

Compara um agente reativo simples com um agente baseado em modelo
no problema do Mundo do Aspirador de Pó.
"""

from typing import Any, Dict, List, Optional, Tuple
import random
import tkinter as tk

import numpy as np

# Estados possíveis de uma célula da grade.
EMPTY: int = 0
DIRT: int = 1
OBSTACLE: int = -1

# Ações possíveis dos agentes.
ACTION_ASPIRAR: str = "Aspirar"
ACTION_MOVER_NORTE: str = "Mover(Norte)"
ACTION_MOVER_SUL: str = "Mover(Sul)"
ACTION_MOVER_LESTE: str = "Mover(Leste)"
ACTION_MOVER_OESTE: str = "Mover(Oeste)"
ACTION_PARAR: str = "Permanecer_Parado"

# Cada movimento é representado por (dy, dx).
MOVIMENTOS: Dict[str, Tuple[int, int]] = {
    ACTION_MOVER_NORTE: (-1, 0),
    ACTION_MOVER_SUL: (1, 0),
    ACTION_MOVER_LESTE: (0, 1),
    ACTION_MOVER_OESTE: (0, -1),
}
```

```

class Ambiente:
    """Representa a grade, suas transições e sua pontuação."""

    def __init__(
        self,
        n: int = 30,
        p_dirt: float = 0.2,
        p_obs: float = 0.15,
        seed: Optional[int] = None,
    ) -> None:
        """Cria uma grade com sujeira e obstáculos aleatórios."""
        self.n: int = n
        self.rng: np.random.Generator = np.random.default_rng(seed)
        self.grid: np.ndarray = np.zeros((self.n, self.n), dtype=int)

        rand_vals: np.ndarray = self.rng.random((self.n, self.n))
        self.grid[rand_vals < p_obs] = OBSTACLE

        dirt_mask = (
            (rand_vals >= p_obs)
            & (rand_vals < p_obs + p_dirt)
        )
        self.grid[dirt_mask] = DIRT

        self.total_dirt: int = int(np.sum(self.grid == DIRT))
        self.aspiradas: int = 0
        self.pontuacao: int = 0

    def get_posicao_valida(self) -> Tuple[int, int]:
        """Retorna uma posição que não contém obstáculo."""
        livres = np.argwhere(self.grid != OBSTACLE)
        escolha = self.rng.choice(len(livres))
        return tuple(livres[escolha])

    def obter_percepcao(self, pos: Tuple[int, int]) -> Dict[str, Any]:
        """Retorna a percepção local disponível ao agente."""
        y, x = pos
        sujo = bool(self.grid[y, x] == DIRT)
        adjacencias: Dict[str, bool] = {}

        for acao, (dy, dx) in MOVIMENTOS.items():
            ny, nx = y + dy, x + dx

            if 0 <= ny < self.n and 0 <= nx < self.n:
                adjacencias[acao] = bool(
                    self.grid[ny, nx] != OBSTACLE
                )

```

```

        else:
            adjacencias[acao] = False

    return {
        "posicao": (y, x),
        "sujo": sujo,
        "adjacencias": adjacencias,
    }

def executar_acao(
    self,
    pos: Tuple[int, int],
    acao: str,
) -> Tuple[Tuple[int, int], str]:
    """Executa uma ação e retorna nova posição e mensagem de pontuação."""
    y, x = pos

    if acao == ACTION_ASPIRAR:
        if self.grid[y, x] == DIRT:
            self.grid[y, x] = EMPTY
            self.aspiradas += 1
            self.pontuacao += 10
            return pos, "+10 (Aspirou)"

        return pos, "0 (Aspirou vazio)"

    if acao in MOVIMENTOS:
        dy, dx = MOVIMENTOS[acao]
        ny, nx = y + dy, x + dx

        if (
            0 <= ny < self.n
            and 0 <= nx < self.n
            and self.grid[ny, nx] != OBSTACLE
        ):
            self.pontuacao -= 1
            return (ny, nx), "-1 (Movimento)"

        self.pontuacao -= 5
        return pos, "-5 (Colisão)"

    return pos, "0 (Parado)"

class AgenteReativoSimples:
    """Agente sem memória, baseado em regras condição-ação."""

    def decidir(self, p: Dict[str, Any]) -> Tuple[str, str]:

```

```

"""Escolhe uma ação usando somente a percepção atual."""
if p["sujo"]:
    return ACTION_ASPIRAR, "Célula suja -> Regra: ASPIRAR"

livres = [
    acao
    for acao, disponivel in p["adjacencias"].items()
    if disponivel
]

if livres:
    escolha = random.choice(livres)
    return escolha, f"Aleatório entre livres: {escolha}"

return ACTION_PARAR, "Sem saídas -> PARAR"

```

```

class AgenteBaseadoEmModelo:
    """Agente com uma matriz interna que registra visitas."""

    def __init__(self, n: int = 30) -> None:
        self.n: int = n
        self.visitas: np.ndarray = np.zeros((n, n), dtype=int)

    def decidir(self, p: Dict[str, Any]) -> Tuple[str, str]:
        """Escolhe sujeira ou a vizinha livre menos visitada."""
        y, x = p["posicao"]
        self.visitas[y, x] += 1

        if p["sujo"]:
            return ACTION_ASPIRAR, "Célula suja -> ASPIRAR"

        livres = [
            acao
            for acao, disponivel in p["adjacencias"].items()
            if disponivel
        ]

        if not livres:
            return ACTION_PARAR, "Sem opções -> PARAR"

        candidatos: List[str] = []
        menor_visitado: int = np.iinfo(np.int32).max

        for acao in livres:
            dy, dx = MOVIMENTOS[acao]
            ny, nx = y + dy, x + dx
            visitas_vizinha = int(self.visitas[ny, nx])

```

```

        if visitas_vizinha < menor_visitas:
            menor_visitas = visitas_vizinha
            candidatos = [acao]
        elif visitas_vizinha == menor_visitas:
            candidatos.append(acao)

    escolha = random.choice(candidatos)
    return (
        escolha,
        f"Célula com menor visitas ({menor_visitas}): {escolha}",
    )

```

```

class InterfaceGrafica:
    """Janela Tkinter que executa e exibe os dois agentes."""

    def __init__(
        self,
        root: tk.Tk,
        n: int = 30,
        t_max: int = 100,
        seed: int = 42,
    ) -> None:
        self.root: tk.Tk = root
        self.root.title("Mundo do Aspirador NxN - Simulação PEAS")
        self.root.configure(bg="#1E1E1E")

        self.n: int = n
        self.t_max: int = t_max
        self.passo: int = 0
        self.rodando: bool = False
        self.cell_sz: int = 14
        canvas_dim = self.n * self.cell_sz

        self.env_r = Ambiente(n=n, seed=seed)
        self.agente_r = AgenteReativoSimples()
        self.pos_r = self.env_r.get_posicao_valida()

        self.env_m = Ambiente(n=n, seed=seed)
        self.agente_m = AgenteBaseadoEmModelo(n=n)
        self.pos_m = self.pos_r

        top_frame = tk.Frame(root, bg="#2D2D30", pady=8)
        top_frame.pack(fill=tk.X)

        self.lbl_status = tk.Label(
            top_frame,

```

```

        text=f"Passo: 0/{self.t_max}",
        fg="white",
        bg="#2D2D30",
        font=("Consolas", 11, "bold"),
    )
    self.lbl_status.pack(side=tk.LEFT, padx=15)

    self.btn_play = tk.Button(
        top_frame,
        text="Iniciar / Pausar",
        command=self.alternar_execucao,
        bg="#007ACC",
        fg="white",
        relief=tk.FLAT,
    )
    self.btn_play.pack(side=tk.LEFT, padx=5)

    self.btn_step = tk.Button(
        top_frame,
        text="Passo a Passo",
        command=self.executar_passo,
        bg="#3E3E42",
        fg="white",
        relief=tk.FLAT,
    )
    self.btn_step.pack(side=tk.LEFT, padx=5)

    self.btn_reset = tk.Button(
        top_frame,
        text="Recomeçar",
        command=self.reiniciar_simulacao,
        bg="#D9534F",
        fg="white",
        relief=tk.FLAT,
    )
    self.btn_reset.pack(side=tk.LEFT, padx=5)

    center_frame = tk.Frame(root, bg="#1E1E1E")
    center_frame.pack(pady=5)

    frame_r = tk.Frame(center_frame, bg="#1E1E1E")
    frame_r.grid(row=0, column=0, padx=10)

    self.lbl_r = tk.Label(
        frame_r,
        text="AGENTE REATIVO SIMPLES\nPontos: 0 | Limpeza: 0.0%",
        fg="#FF6B6B",
        bg="#1E1E1E",
    )

```

```

        font=("Consolas", 10, "bold"),
    )
    self.lbl_r.pack()

    self.canvas_r = tk.Canvas(
        frame_r,
        width=canvas_dim,
        height=canvas_dim,
        bg="white",
        highlightthickness=1,
    )
    self.canvas_r.pack()

    frame_m = tk.Frame(center_frame, bg="#1E1E1E")
    frame_m.grid(row=0, column=1, padx=10)

    self.lbl_m = tk.Label(
        frame_m,
        text="AGENTE BASEADO EM MODELO\nPontos: 0 | Limpeza: 0.0%",
        fg="#4D96FF",
        bg="#1E1E1E",
        font=("Consolas", 10, "bold"),
    )
    self.lbl_m.pack()

    self.canvas_m = tk.Canvas(
        frame_m,
        width=canvas_dim,
        height=canvas_dim,
        bg="white",
        highlightthickness=1,
    )
    self.canvas_m.pack()

    log_frame = tk.Frame(root, bg="#1E1E1E")
    log_frame.pack(fill=tk.BOTH, expand=True, padx=10, pady=5)

    sub_log_r = tk.Frame(log_frame, bg="#1E1E1E")
    sub_log_r.pack(
        side=tk.LEFT,
        fill=tk.BOTH,
        expand=True,
        padx=5,
    )
    tk.Label(
        sub_log_r,
        text="Log de Ações (Reativo)",
        fg="#FF6B6B",
    )

```

```

        bg="#1E1E1E",
        font=("Consolas", 9, "bold"),
    ).pack(anchor=tk.W)

    self.txt_log_r = tk.Text(
        sub_log_r,
        height=8,
        bg="#252526",
        fg="#CCCCCC",
        font=("Consolas", 8),
        relief=tk.FLAT,
    )
    self.txt_log_r.pack(fill=tk.BOTH, expand=True)

    sub_log_m = tk.Frame(log_frame, bg="#1E1E1E")
    sub_log_m.pack(
        side=tk.RIGHT,
        fill=tk.BOTH,
        expand=True,
        padx=5,
    )
    tk.Label(
        sub_log_m,
        text="Log de Ações (Modelo com Memória)",
        fg="#4D96FF",
        bg="#1E1E1E",
        font=("Consolas", 9, "bold"),
    ).pack(anchor=tk.W)

    self.txt_log_m = tk.Text(
        sub_log_m,
        height=8,
        bg="#252526",
        fg="#CCCCCC",
        font=("Consolas", 8),
        relief=tk.FLAT,
    )
    self.txt_log_m.pack(fill=tk.BOTH, expand=True)

    self.desenhar_grade(self.canvas_r, self.env_r, self.pos_r, "#FF6B6B")
    self.desenhar_grade(self.canvas_m, self.env_m, self.pos_m, "#4D96FF")

def reiniciar_simulacao(self) -> None:
    """Cria dois novos ambientes usando uma nova seed."""
    self.rodando = False
    self.passo = 0
    nova_seed = random.randint(1, 99999)

```

```

self.env_r = Ambiente(n=self.n, seed=nova_seed)
self.agente_r = AgenteReativoSimples()
self.pos_r = self.env_r.get_posicao_valida()

self.env_m = Ambiente(n=self.n, seed=nova_seed)
self.agente_m = AgenteBaseadoEmModelo(n=self.n)
self.pos_m = self.pos_r

self.txt_log_r.delete("1.0", tk.END)
self.txt_log_m.delete("1.0", tk.END)

self.lbl_status.config(text=f"Passo: 0/{self.t_max}")
self.lbl_r.config(
    text="AGENTE REATIVO SIMPLES\nPontos: 0 | Limpeza: 0.0%"
)
self.lbl_m.config(
    text="AGENTE BASEADO EM MODELO\nPontos: 0 | Limpeza: 0.0%"
)

self.desenhar_grade(self.canvas_r, self.env_r, self.pos_r, "#FF6B6B")
self.desenhar_grade(self.canvas_m, self.env_m, self.pos_m, "#4D96FF")

def desenhar_grade(
    self,
    canvas: tk.Canvas,
    env: Ambiente,
    pos_agente: Tuple[int, int],
    cor_agente: str,
) -> None:
    """Desenha células, sujeira, obstáculos e agente."""
    canvas.delete("all")
    cs = self.cell_sz

    for y in range(self.n):
        for x in range(self.n):
            val = env.grid[y, x]
            cor = "#FFFFFF"

            if val == OBSTACLE:
                cor = "#333333"
            elif val == DIRT:
                cor = "#E5A93C"

            canvas.create_rectangle(
                x * cs,
                y * cs,
                (x + 1) * cs,
                (y + 1) * cs,

```

```

        fill=cor,
        outline="#EEEEEE",
    )

    if val == DIRT:
        canvas.create_oval(
            x * cs + 4,
            y * cs + 4,
            (x + 1) * cs - 4,
            (y + 1) * cs - 4,
            fill="#B27300",
            outline="",
        )

```

```

ay, ax = pos_agente
canvas.create_oval(
    ax * cs + 2,
    ay * cs + 2,
    (ax + 1) * cs - 2,
    (ay + 1) * cs - 2,
    fill=cor_agente,
    outline="white",
    width=2,
)

```

```

def registrar_log(self, text_widget: tk.Text, msg: str) -> None:
    """Adiciona uma mensagem e rola o log para o final."""
    text_widget.insert(tk.END, msg + "\n")
    text_widget.see(tk.END)

```

```

def executar_passo(self) -> None:
    """Executa um passo para cada agente."""
    if self.passo >= self.t_max:
        self.rodando = False
        return

    self.passo += 1

    percepcao_r = self.env_r.obter_percepcao(self.pos_r)
    acao_r, explicacao_r = self.agente_r.decidir(percepcao_r)
    self.pos_r, score_r = self.env_r.executar_acao(
        self.pos_r,
        acao_r,
    )

```

```

taxa_r = (
    self.env_r.aspiradas / self.env_r.total_dirt * 100
    if self.env_r.total_dirt > 0

```

```

        else 0
    )
    self.lbl_r.config(
        text=(
            "AGENTE REATIVO SIMPLES\n"
            f"Pontos: {self.env_r.pontuacao} | "
            f"Limpeza: {taxa_r:.1f}%"
        )
    )
    self.registrar_log(
        self.txt_log_r,
        (
            f"[T={self.passo:03d}] "
            f"Pos: {percepcao_r['posicao']} | "
            f"Decisão: {explicacao_r} | "
            f"Custo: {score_r}"
        ),
    )
    self.desenhar_grade(
        self.canvas_r,
        self.env_r,
        self.pos_r,
        "#FF6B6B",
    )

    percepcao_m = self.env_m.obter_percepcao(self.pos_m)
    acao_m, explicacao_m = self.agente_m.decidir(percepcao_m)
    self.pos_m, score_m = self.env_m.executar_acao(
        self.pos_m,
        acao_m,
    )

    taxa_m = (
        self.env_m.aspiradas / self.env_m.total_dirt * 100
        if self.env_m.total_dirt > 0
        else 0
    )
    self.lbl_m.config(
        text=(
            "AGENTE BASEADO EM MODELO\n"
            f"Pontos: {self.env_m.pontuacao} | "
            f"Limpeza: {taxa_m:.1f}%"
        )
    )
    self.registrar_log(
        self.txt_log_m,
        (
            f"[T={self.passo:03d}] "

```

```

        f"Pos: {percepcao_m['posicao']} | "
        f"Decisão: {explicacao_m} | "
        f"Custo: {score_m}"
    ),
)
self.desenhar_grade(
    self.canvas_m,
    self.env_m,
    self.pos_m,
    "#4D96FF",
)

self.lbl_status.config(
    text=f"Passo: {self.passo}/{self.t_max}"
)

def loop_animacao(self) -> None:
    """Executa passos repetidos sem bloquear a janela."""
    if self.rodando and self.passo < self.t_max:
        self.executar_passo()
        self.root.after(100, self.loop_animacao)
    elif self.passo >= self.t_max:
        self.rodando = False

def alternar_execucao(self) -> None:
    """Inicia ou pausa a animação."""
    self.rodando = not self.rodando

    if self.rodando:
        self.loop_animacao()

if __name__ == "__main__":
    janela = tk.Tk()
    app = InterfaceGrafica(janela, n=30, t_max=100, seed=42)
    janela.mainloop()

```

4. Construção do código, bloco por bloco

4.1 Docstring do módulo

O arquivo começa com uma string entre três aspas:

Python

```
"""Simulação comparativa de agentes inteligentes."""
```

Essa string é chamada de **docstring**. Ela documenta o propósito do arquivo. Não é uma instrução que movimenta o agente. É uma explicação para quem lê o programa e também pode ser consultada por ferramentas de documentação.

Uma analogia é a etiqueta de uma caixa. A etiqueta não altera o conteúdo da caixa, mas informa o que existe dentro dela.

4.2 Importações

Python

```
from typing import Any, Dict, List, Optional, Tuple
import random
import tkinter as tk
import numpy as np
```

A instrução `from typing import ...` importa nomes usados nas anotações de tipos:

- `Any` significa “qualquer tipo” ;
- `Dict` representa dicionários;
- `List` representa listas;
- `Optional` indica que um valor pode existir ou ser `None` ;
- `Tuple` representa tuplas.

As anotações não são obrigatórias para o Python executar o programa. Elas funcionam como placas de orientação para o programador.

`import random` importa a biblioteca de escolhas aleatórias do Python. Ela é usada para desempatar movimentos ou escolher uma saída livre.

`import tkinter as tk` importa Tkinter e cria o apelido `tk` . Assim, escrevemos `tk.Tk()` em vez de `tkinter.Tk()` .

`import numpy as np` importa NumPy com o apelido usual `np` . NumPy é usado para representar grades e matrizes de visitas.

4.3 Constantes de estado

Python

```
EMPTY: int = 0
DIRT: int = 1
OBSTACLE: int = -1
```

Essas constantes representam os estados físicos possíveis de uma célula:

Nome	Valor	Significado
EMPTY	0	Célula navegável e sem sujeira
DIRT	1	Célula navegável com sujeira
OBSTACLE	-1	Célula bloqueada

A anotação `:int` informa que esperamos um número inteiro.

Por que usar nomes em vez de escrever diretamente `0`, `1` e `-1`? Porque isto é mais legível:

Python

```
if valor == OBSTACLE:
```

do que isto:

Python

```
if valor == -1:
```

O primeiro caso explica a intenção. O segundo exige que o leitor memorize o significado de `-1`.

4.4 Ações discretas

Python

```
ACTION_ASPIRAR: str = "Aspirar"
ACTION_MOVER_NORTE: str = "Mover(Norte)"
...
```

Uma ação discreta é uma opção separada, como um botão de controle. O agente não escolhe “mover 3,72 unidades”. Ele escolhe uma das opções nomeadas.

As ações de movimento são strings porque os nomes aparecem nos logs. A constante `ACTION_PARAR` representa a decisão de não sair do lugar.

Dicionário MOVIMENTOS

Python

```
MOVIMENTOS = {  
    ACTION_MOVER_NORTE: (-1, 0),  
    ACTION_MOVER_SUL: (1, 0),  
    ACTION_MOVER_LESTE: (0, 1),  
    ACTION_MOVER_OESTE: (0, -1),  
}
```

A posição é guardada como `(y, x)` :

- `y` é a linha;
- `x` é a coluna.

Por isso:

- norte diminui a linha: `(-1, 0)` ;
- sul aumenta a linha: `(1, 0)` ;
- leste aumenta a coluna: `(0, 1)` ;
- oeste diminui a coluna: `(0, -1)` .

Se o agente está em `(4, 7)` e escolhe norte, a nova posição é `(3, 7)` . Se escolhe leste, a nova posição é `(4, 8)` .

O dicionário funciona como um tradutor entre nomes e deslocamentos.

4.5 Classe Ambiente

Python

```
class Ambiente:
```

Uma **classe** é um molde para criar objetos. O molde `Ambiente` descreve tudo que um ambiente sabe fazer: criar uma grade, fornecer percepções e executar ações.

Um objeto criado a partir da classe é chamado de **instância**. Neste programa, `self.env_r` e `self.env_m` são duas instâncias de `Ambiente` .

Método `__init__`

Python

```
def __init__(self, n=30, p_dirt=0.2, p_obs=0.15, seed=None):
```

`__init__` é executado automaticamente quando criamos um objeto. Os parâmetros são:

- `n` : largura e altura da grade;
- `p_dirt` : probabilidade de sujeira;
- `p_obs` : probabilidade de obstáculo;
- `seed` : semente aleatória.

`self` representa o próprio objeto. É como dizer “a grade pertencente a este ambiente” .

Python

```
self.n = n
```

Guarda o tamanho dentro do objeto. Depois, outros métodos podem acessá-lo como `self.n` .

Python

```
self.rng = np.random.default_rng(seed)
```

Cria um gerador pseudoaleatório do NumPy. “Pseudoaleatório” significa que os valores parecem aleatórios, mas são produzidos por um algoritmo determinístico. Se a mesma seed for usada, a sequência pode ser repetida.

Python

```
self.grid = np.zeros((self.n, self.n), dtype=int)
```

Cria uma matriz cheia de zeros. O formato `(n, n)` significa `n` linhas e `n` colunas. Inicialmente, todas as células são `EMPTY` .

`dtype=int` pede números inteiros.

Criando obstáculos e sujeira

Python

```
rand_vals = self.rng.random((self.n, self.n))
```

Cria uma matriz de números entre 0 e 1. Cada célula recebe um valor aleatório.

A máscara de obstáculos é:

Python

```
self.grid[rand_vals < p_obs] = OBSTACLE
```

A expressão `rand_vals < p_obs` gera uma matriz de valores booleanos. Se `p_obs` for 0.15, aproximadamente 15% dos valores serão menores que 0.15. Essas posições recebem -1.

A máscara de sujeira é:

Python

```
dirt_mask = (  
    (rand_vals >= p_obs)  
    & (rand_vals < p_obs + p_dirt)  
)
```

O símbolo `&` significa “e” entre matrizes booleanas. Uma célula recebe sujeira quando:

1. não foi escolhida para ser obstáculo; e
2. o valor aleatório está dentro da faixa de sujeira.

Com `p_obs=0.15` e `p_dirt=0.2`, a sujeira ocupa a faixa de 0.15 até 0.35. Assim, as probabilidades são aproximadamente:

- 15% de obstáculos;
- 20% de sujeira;
- 65% de células vazias.

Python

```
self.grid[dirt_mask] = DIRT
```

Usa a máscara para colocar 1 nas posições selecionadas.

Métricas do ambiente

Python

```
self.total_dirt = int(np.sum(self.grid == DIRT))
```

`self.grid == DIRT` produz uma máscara indicando onde há sujeira. `np.sum` conta quantos valores verdadeiros existem. `int` converte o resultado para inteiro comum.

Python

```
self.aspiradas = 0
self.pontuacao = 0
```

Esses contadores começam em zero. `aspiradas` conta células limpas. `pontuacao` soma recompensas e penalidades.

Método `get_posicao_valida`

Python

```
def get_posicao_valida(self):
    livres = np.argwhere(self.grid != OBSTACLE)
    escolha = self.rng.choice(len(livres))
    return tuple(livres[escolha])
```

`self.grid != OBSTACLE` cria uma máscara para todas as células que não são obstáculos.

`np.argwhere` devolve as coordenadas dessas células. Um resultado possível seria:

Python

```
[[0, 1], [0, 2], [1, 0]]
```

`len(livres)` informa quantas posições existem. `self.rng.choice` escolhe um índice aleatório. Por fim, `tuple` transforma a coordenada em uma tupla, como `(1, 0)`.

A função garante apenas que a posição inicial não seja um obstáculo. Ela pode conter sujeira, o que é útil para testar a reação imediata do agente.

Método `obter_percepcao`

Python

```
y, x = pos
```

Se `pos` for `(4, 7)`, `y` recebe `4` e `x` recebe `7`. Essa operação é chamada de desempacotamento de tupla.

Python

```
sujo = bool(self.grid[y, x] == DIRT)
```

Lê a célula atual e transforma o resultado em `True` ou `False` .

Python

```
adjacencias = {}
```

Cria um dicionário vazio para registrar se cada movimento é possível.

Python

```
for acao, (dy, dx) in MOVIMENTOS.items():
```

Percorre todas as ações de movimento. Em cada repetição, recebe o nome da ação e seu vetor.

Python

```
ny, nx = y + dy, x + dx
```

Calcula a coordenada vizinha. Se a posição atual for `(4, 7)` e o movimento for norte, a vizinha será `(3, 7)` .

Python

```
if 0 <= ny < self.n and 0 <= nx < self.n:
```

Verifica se a vizinha está dentro da grade. Essa condição evita acessar uma linha ou coluna inexistente.

Python

```
adjacencias[acao] = bool(self.grid[ny, nx] != OBSTACLE)
```

Se está dentro da grade, verifica se não é obstáculo. Uma célula com sujeira continua sendo navegável.

Python

```
else:  
    adjacencias[acao] = False
```

Se a vizinha estiver fora da grade, o movimento é marcado como inválido.

O retorno é um dicionário como:

Python

```
{
    "posicao": (4, 7),
    "sujo": False,
    "adjacencias": {
        "Mover(Norte)": True,
        "Mover(Sul)": True,
        "Mover(Leste)": False,
        "Mover(Oeste)": True,
    },
}
```

O agente não recebe a grade inteira. Ele recebe apenas essa visão local. Isso caracteriza **observabilidade parcial**.

Método `executar_acao`

O método recebe posição e ação e devolve nova posição e uma mensagem.

Python

```
y, x = pos
```

Se a ação for aspirar:

Python

```
if acao == ACTION_ASPIRAR:
```

O ambiente verifica a célula atual:

Python

```
if self.grid[y, x] == DIRT:
```

Se houver sujeira, ela é substituída por `EMPTY` :

Python

```
self.grid[y, x] = EMPTY
```

Depois, os contadores são atualizados:

Python

```
self.aspiradas += 1
self.pontuacao += 10
```

O método retorna a mesma posição, porque aspirar não movimenta o agente, e a mensagem +10 (Aspirou) .

Se a célula estiver vazia, a posição também não muda e a pontuação não é alterada.

Para movimentos, o método consulta o vetor:

Python

```
dy, dx = MOVIMENTOS[acao]
ny, nx = y + dy, x + dx
```

Em seguida, verifica limites e obstáculos. Se o movimento for válido, subtrai um ponto e retorna a posição nova. Se for inválido, subtrai cinco pontos e retorna a posição antiga.

A posição antiga é retornada na colisão porque o agente não atravessa uma parede.

4.6 Classe AgenteReativoSimples

Python

```
class AgenteReativoSimples:
```

Esse agente implementa regras do tipo **condição → ação**.

A regra principal é:

Python

```
if p["sujo"]:
    return ACTION_ASPIRAR, "Célula suja -> Regra: ASPIRAR"
```

Se o sensor informar sujeira, a prioridade é aspirar. O `return` encerra o método imediatamente.

Quando não há sujeira, o agente filtra as ações disponíveis:

Python

```
livres = [
    acao
    for acao, disponivel in p["adjacencias"].items()
    if disponivel
]
```

Essa é uma **compreensão de lista**. Ela pode ser lida assim:

Para cada ação e disponibilidade no dicionário, mantenha a ação quando a disponibilidade for verdadeira.

Se houver saídas:

Python

```
escolha = random.choice(livres)
```

uma delas é sorteada.

O agente não consulta o histórico. Portanto, visitar uma célula dez vezes não altera sua decisão futura. Essa simplicidade é uma vantagem para a implementação, mas pode causar repetição de trajetórias.

4.7 Classe AgenteBaseadoEmModelo

Esse agente possui um estado interno:

Python

```
self.visitas = np.zeros((n, n), dtype=int)
```

A matriz começa com zeros. Se a célula (3,4) for visitada três vezes, teremos:

Python

```
self.visitas[3, 4] == 3
```

No início da decisão:

Python

```
y, x = p["posicao"]
self.visitas[y, x] += 1
```

O agente registra que está passando pela posição atual.

A prioridade da sujeira permanece:

Python

```
if p["sujo"]:
    return ACTION_ASPIRAR, "Célula suja -> ASPIRAR"
```

Depois, ele obtém as vizinhas livres. Se não houver nenhuma, permanece parado.

A heurística começa com:

Python

```
candidatos = []
menor_visitas = np.iinfo(np.int32).max
```

`candidatos` armazenará as ações empatadas. O valor inicial de `menor_visitas` é um inteiro muito grande, para que qualquer valor real encontrado seja menor.

Para cada ação livre:

Python

```
dy, dx = MOVIMENTOS[acao]
ny, nx = y + dy, x + dx
visitas_vizinha = int(self.visitas[ny, nx])
```

O agente transforma a ação em coordenada e consulta a matriz de visitas.

Se encontrar um valor menor:

Python

```
if visitas_vizinha < menor_visitas:
    menor_visitas = visitas_vizinha
    candidatos = [acao]
```

A menor pontuação é atualizada e a lista de candidatos é substituída.

Se encontrar empate:

Python

```
elif visitas_vizinha == menor_visitas:
    candidatos.append(acao)
```

A nova ação entra na lista. Ao final, uma ação entre as empatadas é sorteada. Assim, o agente evita uma regra arbitrária quando duas direções são igualmente boas.

Importante: essa estratégia é **local**. O agente olha apenas para vizinhos imediatos. Ele não calcula o caminho completo até a sujeira mais próxima e não garante a limpeza total.

4.8 Classe InterfaceGrafica

A classe gráfica conecta o modelo à tela. Ela não é responsável por decidir qual agente é mais inteligente. Sua função é mostrar o que acontece.

Inicialização da janela

Python

```
self.root = root
self.root.title("Mundo do Aspirador NxN - Simulação PEAS")
self.root.configure(bg="#1E1E1E")
```

`root` é a janela criada por Tkinter. `title` define o texto da barra de título. `configure` aplica a cor de fundo.

Os atributos de controle são:

Python

```
self.n = n
self.t_max = t_max
self.passo = 0
self.rodando = False
self.cell_sz = 14
```

- `n` é o tamanho da grade;
- `t_max` é o número máximo de passos;
- `passo` é o passo atual;
- `rodando` informa se a animação está ativa;
- `cell_sz` é o tamanho visual de cada célula em pixels.

Dois ambientes equivalentes

Python

```
self.env_r = Ambiente(n=n, seed=seed)
self.env_m = Ambiente(n=n, seed=seed)
```

Como a mesma seed é usada, as grades começam iguais. Um ambiente é independente do outro, então a sujeira removida por um agente não afeta o outro.

Python

```
self.pos_r = self.env_r.get_posicao_valida()
self.pos_m = self.pos_r
```

A posição inicial é escolhida uma vez e copiada para o segundo agente. Isso garante que a comparação comece do mesmo ponto.

Frames, labels e botões

Um `Frame` é um contêiner visual. Ele organiza outros elementos.

Python

```
top_frame = tk.Frame(root, bg="#2D2D30", pady=8)
top_frame.pack(fill=tk.X)
```

O frame superior recebe o status e os botões. `pack` posiciona o elemento. `fill=tk.X` permite ocupar a largura.

Um `Label` exibe texto:

Python

```
self.lbl_status = tk.Label(top_frame, text="Passo: 0/100")
```

Um `Button` é clicável. O parâmetro `command` recebe uma função sem parênteses. Isso significa que a função será executada quando o botão for clicado:

Python

```
command=self.executar_passo
```

Se escrevêssemos `command=self.executar_passo()`, a função seria executada imediatamente durante a construção da janela, o que não é desejado.

Os três botões são:

- iniciar/pausar;

- passo a passo;
- recomençar.

Canvas das grades

Canvas é uma área de desenho bidimensional. O programa cria um canvas para cada agente:

Python

```
self.canvas_r = tk.Canvas(...)  
self.canvas_m = tk.Canvas(...)
```

A separação visual permite comparar trajetórias lado a lado.

Logs

Text cria uma área editável de texto. Aqui ela funciona como um terminal visual.

Cada log registra:

- número do passo;
- posição percebida;
- justificativa da decisão;
- resultado da ação.

reiniciar_simulacao

O método interrompe a animação, zera o contador e cria uma nova seed:

Python

```
self.rodando = False  
self.passo = 0  
nova_seed = random.randint(1, 99999)
```

Depois, recria os dois ambientes e os dois agentes. Apaga os logs com:

Python

```
self.txt_log_r.delete("1.0", tk.END)
```

Em widgets Text, "1.0" significa linha 1, coluna 0. tk.END representa o fim do conteúdo. Por fim, atualiza textos e redesenha as grades.

desenhar_grade

O primeiro comando limpa o desenho anterior:

Python

```
canvas.delete("all")
```

Depois, dois `for` percorrem todas as linhas e colunas:

Python

```
for y in range(self.n):  
    for x in range(self.n):
```

A cor começa como branca para célula vazia. Obstáculos ficam cinza-escuro e sujeira fica âmbar.

O retângulo de uma célula é desenhado com coordenadas de tela:

Python

```
canvas.create_rectangle(  
    x * cs, y * cs,  
    (x + 1) * cs, (y + 1) * cs,  
    fill=cor,  
)
```

Se `cs=14` e `x=2`, a célula começa no pixel `28`. A multiplicação converte coordenadas da grade em coordenadas da tela.

A sujeira recebe também um oval menor para parecer um ponto.

No final, o agente é desenhado como um oval maior. A ordem é importante: ele é desenhado depois da grade, portanto aparece por cima das células.

registrar_log

Python

```
text_widget.insert(tk.END, msg + "\n")  
text_widget.see(tk.END)
```

A primeira linha acrescenta a mensagem. A segunda rola automaticamente o widget para mostrar a mensagem mais recente.

executar_passo

Esse é o coração da simulação. Para cada agente, ocorre a sequência:

1. obter percepção;
2. tomar decisão;
3. executar ação;
4. calcular métrica;
5. atualizar label;
6. registrar log;
7. redesenhar grade.

O limite é verificado no começo:

Python

```
if self.passo >= self.t_max:  
    self.rodando = False  
    return
```

Depois, o contador aumenta:

Python

```
self.passo += 1
```

O pipeline reativo é:

Python

```
percepcao_r = self.env_r.obter_percepcao(self.pos_r)  
acao_r, explicacao_r = self.agente_r.decidir(percepcao_r)  
self.pos_r, score_r = self.env_r.executar_acao(  
    self.pos_r,  
    acao_r,  
)
```

A percepção é separada da decisão. Essa separação é importante porque permite trocar o agente sem reescrever o ambiente.

A taxa de limpeza é:

Python

```

taxa_r = (
    self.env_r.aspiradas / self.env_r.total_dirt * 100
    if self.env_r.total_dirt > 0
    else 0
)

```

O teste contra zero evita divisão por zero em uma grade que, por acaso, não tenha sujeira.

O pipeline do agente baseado em modelo repete a mesma estrutura, usando `env_m`, `agente_m` e `pos_m`.

loop_animacao e root.after

Tkinter possui um loop de eventos que precisa continuar livre para responder a cliques e redesenhar a janela. Por isso, não usamos um `while` infinito que bloquearia a interface.

Python

```
self.root.after(100, self.loop_animacao)
```

Esse comando agenda uma nova chamada depois de 100 milissegundos. O método agenda a próxima chamada apenas enquanto `rodando` for verdadeiro e o limite não tiver sido atingido.

alternar_execucao

Python

```
self.rodando = not self.rodando
```

O operador `not` troca `True` por `False` e vice-versa. Se a animação foi ligada, o método chama `loop_animacao`.

Ponto de entrada

Python

```

if __name__ == "__main__":
    janela = tk.Tk()
    app = InterfaceGrafica(janela, n=30, t_max=100, seed=42)
    janela.mainloop()

```

Quando um arquivo Python é executado diretamente, `__name__` vale `"__main__"`. Assim, a janela é criada apenas quando o arquivo é executado, e não quando ele é importado por

outro arquivo.

`tk.Tk()` cria a janela principal. `InterfaceGrafica` monta os elementos. `mainloop()` mantém a janela aberta e processa eventos.

5. Explicação dos conceitos-chave

5.1 Grade $N \times N$

Uma grade $N \times N$ possui N linhas e N colunas. Ela é semelhante a um tabuleiro de xadrez, mas cada quadrado é armazenado como uma posição de uma matriz.

Se $N=3$, as coordenadas são:

Plain Text

```
(0,0) (0,1) (0,2)
(1,0) (1,1) (1,2)
(2,0) (2,1) (2,2)
```

Python e NumPy começam a contar índices em zero.

5.2 Agente reativo simples

Um agente reativo simples responde diretamente ao percepto atual. É parecido com uma lâmpada automática:

| Se detectar escuro, acenda; caso contrário, permaneça apagada.

Ele não pergunta o que aconteceu ontem. No programa, ele aspira se vê sujeira; caso contrário, escolhe uma vizinha livre.

5.3 Agente baseado em modelo

Um agente baseado em modelo mantém uma representação interna de aspectos do mundo. Neste projeto, essa representação é a matriz de visitas.

Ele ainda observa localmente, mas também usa informação acumulada. A memória ajuda a evitar ciclos repetitivos.

5.4 Matriz de calor ou memória de visitas

A matriz de visitas é como um **mapa de calor**. Células muito visitadas possuem valores altos; células pouco exploradas possuem valores baixos.

A implementação não desenha esse mapa na tela, mas os dados existem em `agente_m.visitas`.

Uma possível visualização futura poderia usar cores claras para valores baixos e cores fortes para valores altos.

5.5 Seed

Uma seed é o ponto inicial de um gerador pseudoaleatório. Usar a mesma seed permite repetir o mesmo experimento.

Isso é útil para depuração. Se um comportamento estranho ocorrer na seed `42`, podemos executá-lo novamente e investigá-lo.

A seed não torna todas as escolhas do programa idênticas, porque o código usa tanto o gerador do NumPy quanto o módulo `random`. Ela garante principalmente a mesma grade inicial.

5.6 Percepção parcialmente observável

O agente não conhece a grade inteira. Ele observa apenas a célula atual e a navegabilidade das quatro vizinhas.

É como caminhar em uma casa escura com uma lanterna curta: você vê o local onde está e os espaços imediatamente próximos, mas não enxerga o mapa completo.

5.7 Recompensas e penalidades

Uma recompensa aumenta a pontuação e uma penalidade diminui a pontuação. Essa função transforma objetivos qualitativos em números comparáveis.

A pontuação não é a única métrica. Um agente pode limpar mais sujeira, mas gastar mais passos. Por isso, convém observar simultaneamente:

- pontuação;
- percentual de limpeza;
- quantidade de movimentos;
- quantidade de colisões;
- cobertura espacial.

5.8 Tkinter

Tkinter é a biblioteca gráfica incluída no Python. Ela oferece janelas, botões, textos, rótulos, contêineres e áreas de desenho.

O programa usa:

Widget	Função
Tk	Janela principal
Frame	Contêiner de organização
Label	Texto informativo
Button	Botão clicável
Canvas	Desenho da grade
Text	Área de logs

5.9 NumPy e arrays

Um array NumPy é uma estrutura eficiente para armazenar muitos valores organizados em dimensões.

Uma lista de listas também poderia representar a grade, mas NumPy facilita:

- criar matrizes inteiras;
- gerar números aleatórios em bloco;
- aplicar máscaras booleanas;
- contar valores com `np.sum` ;
- localizar coordenadas com `np.argwhere` .

6. Como testar e modificar

6.1 Executar a simulação

Execute o arquivo:

```
Bash
```

```
python mundo_aspirador.py
```

Na janela:

1. clique em **Passo a Passo** para observar uma decisão por vez;
2. clique em **Iniciar / Pausar** para animar automaticamente;
3. clique em **Recomeçar** para gerar uma nova configuração.

Leia os logs e compare a justificativa dos agentes.

6.2 Alterar o tamanho da grade

No ponto de entrada:

```
Python
```

```
app = InterfaceGrafica(janela, n=30, t_max=100, seed=42)
```

Altere `n`:

```
Python
```

```
app = InterfaceGrafica(janela, n=15, t_max=100, seed=42)
```

Grades menores são mais fáceis de visualizar. Grades maiores permitem trajetórias mais longas, mas podem exigir uma janela maior.

6.3 Alterar o tempo máximo

Para executar 500 passos:

```
Python
```

```
app = InterfaceGrafica(janela, n=30, t_max=500, seed=42)
```

Mais passos dão mais tempo para explorar, mas tornam os logs mais extensos.

6.4 Alterar a seed

Para reproduzir outro mundo:

```
Python
```

```
app = InterfaceGrafica(janela, n=30, t_max=100, seed=123)
```

Compare as seeds 42, 123 e 999.

6.5 Alterar sujeira e obstáculos

Por padrão, a interface usa:

Python

```
Ambiente(n=n, seed=seed)
```

Isso aproveita os valores padrão `p_dirt=0.2` e `p_obs=0.15`. Para personalizá-los, altere a criação dos ambientes:

Python

```
self.env_r = Ambiente(  
    n=n,  
    p_dirt=0.35,  
    p_obs=0.10,  
    seed=seed,  
)
```

Repita os mesmos valores em `env_m` para manter a comparação justa.

Tenha cuidado: se `p_dirt + p_obs` ultrapassar `1.0`, a configuração fica conceitualmente inadequada. Além disso, probabilidades muito altas de obstáculos podem separar a grade em regiões isoladas.

6.6 O que observar na comparação

Faça uma tabela para cada experimento:

Seed	Agente	Pontuação	Limpeza	Observações
42	Reativo	preencher	preencher	preencher
42	Modelo	preencher	preencher	preencher

Não conclua que o agente baseado em modelo sempre vence. Ele usa uma heurística simples e pode gastar movimentos explorando áreas que não têm sujeira. A avaliação

correta exige várias seeds e médias.

6.7 Um detalhe importante sobre a aleatoriedade

A grade usa o gerador do NumPy, enquanto escolhas de ações usam `random.choice`. Para experimentos totalmente reproduzíveis, inicialize também o módulo `random`:

```
Python
```

```
random.seed(seed)
```

Isso pode ser feito no início de `InterfaceGrafica.__init__` ou antes da criação dos agentes. Em uma comparação científica, também é recomendável registrar a seed usada em cada execução.

7. Desafios propostos

Desafio 1 — Criar um agente baseado em objetivo

Crie uma classe `AgenteBaseadoEmObjetivo`. Ao detectar sujeira apenas localmente, ele deve aspirar. Quando não houver sujeira, tente escolher movimentos que aproximem o agente de uma posição-alvo.

Uma primeira versão pode usar uma posição fixa, como o centro da grade. Depois, você pode criar um objetivo dinâmico, como “visitar todas as células navegáveis”.

Desafio 2 — Adicionar um placar final comparativo

Inclua um botão ou uma mensagem ao final com:

- pontuação de cada agente;
- quantidade de sujeira aspirada;
- percentual de limpeza;
- número de passos;
- vencedor por pontuação;
- vencedor por limpeza.

Lembre-se de tratar empates.

Desafio 3 — Salvar logs em arquivo .txt

Crie uma função como:

Python

```
def salvar_log(self, text_widget, nome_arquivo):
    conteudo = text_widget.get("1.0", tk.END)
    with open(nome_arquivo, "w", encoding="utf-8") as arquivo:
        arquivo.write(conteudo)
```

O bloco `with open(...)` abre o arquivo e garante que ele seja fechado corretamente.

Desafio 4 — Modificar as cores

Troque as cores em `desenhar_grade`:

Python

```
if val == OBSTACLE:
    cor = "#444444"
elif val == DIRT:
    cor = "#F4C542"
```

Também altere `cor_agente` para mudar a cor dos robôs.

Desafio 5 — Mostrar o mapa de calor

Use `agente_m.visitas` para alterar a cor de células visitadas. Uma possibilidade é desenhar células visitadas com tons diferentes. Quanto maior o número de visitas, mais escuro seria o preenchimento.

Desafio 6 — Medir colisões

Adicione um atributo `colisoes` ao ambiente:

Python

```
self.colisoes = 0
```

No bloco de movimento inválido:

Python

```
self.colisoos += 1
self.pontuacao -= 5
```

Depois, mostre essa métrica no cabeçalho ou no relatório final.

8. Glossário

Termo	Explicação simples
Ação	Operação escolhida pelo agente, como mover ou aspirar.
Agente	Sistema que percebe um ambiente e atua nele.
Array	Estrutura NumPy que guarda valores organizados em uma ou mais dimensões.
Atributo	Dado guardado dentro de um objeto, como <code>self.pontuacao</code> .
Classe	Molde usado para criar objetos.
Coordenada	Par que identifica uma posição, como <code>(linha, coluna)</code> .
Dicionário	Estrutura que associa chaves a valores.
Docstring	Texto usado para documentar código Python.
Estado	Situação atual de uma célula ou do ambiente.
Evento	Ocorrência processada pela interface, como um clique.
Função	Bloco reutilizável de instruções.
Heurística	Regra prática para escolher uma ação sem explorar todas as possibilidades.
Instância	Objeto criado a partir de uma classe.
Máscara booleana	Matriz de <code>True</code> e <code>False</code> usada para selecionar valores.
Método	Função definida dentro de uma classe.

Matriz	Estrutura retangular composta por linhas e colunas.
PEAS	Performance, Environment, Actuators e Sensors.
Percepção	Informações recebidas pelos sensores.
Probabilidade	Medida entre 0 e 1 que expressa chance de ocorrência.
Recompensa	Pontos positivos recebidos por uma ação.
Seed	Valor inicial que permite repetir uma sequência pseudoaleatória.
Sensor	Mecanismo que fornece informação sobre o ambiente.
Tupla	Coleção ordenada, geralmente usada para coordenadas.
Widget	Elemento visual de uma interface Tkinter.

9. Resumo do que foi aprendido

Neste projeto, construímos uma simulação completa de agentes inteligentes em uma grade bidimensional. A grade usa valores inteiros para representar células vazias, sujeira e obstáculos.

Aprendemos que uma classe é um molde para objetos e que seus métodos representam comportamentos. A classe `Ambiente` cria o mundo, fornece percepções e aplica ações. O agente reativo simples toma decisões imediatas sem memória. O agente baseado em modelo mantém uma matriz de visitas e escolhe vizinhas menos exploradas.

Também vimos como o formalismo PEAS descreve desempenho, ambiente, atuadores e sensores. A pontuação torna possível comparar ações. A seed ajuda a reproduzir experimentos. A observabilidade parcial limita o que o agente consegue saber.

Na parte gráfica, usamos Tkinter para criar uma janela, botões, labels, canvases e logs. O método `root.after` permite animar a simulação sem congelar a interface.

O aprendizado mais importante é a separação entre três responsabilidades:

1. **Ambiente:** representa o mundo e suas regras.

2. **Agente:** decide o que fazer.
3. **Interface:** mostra o que aconteceu.

Essa separação torna o programa mais fácil de testar, modificar e ampliar.

Referências

- [1] Python Documentation
- [2] Python tkinter — Python Documentation
- [3] NumPy Documentation
- [4] Python random — Python Documentation
- [5] Artificial Intelligence: A Modern Approach
- [6] Python Classes — Python Documentation
- [7] Python typing — Python Documentation
- [8] Python open Function — Python Documentation
- [9] PDF fornecido pelo usuário: Código comentado da simulação